

lambda calcolo e programmazione funzionale

funzioni di ordine superiore

- nel lambda calcolo una funzione ha esattamente una variabile legata e può essere applicata ad un solo argomento
- le funzioni con + argomenti devono quindi presentare un annidamento di λ -ascolazioni, ovvero prendere un argomento alla volta
- una funzione che accetta 2 argomenti è così

$\lambda x. (\lambda y. xy)$

- Matematicamente è come passare una funzione $f: (x \times y) \rightarrow z$ a una funzione $g: x \rightarrow (y \rightarrow z)$

g è una funzione che prende un argomento x e restituisce una funzione che mette in relazione y e z

- questa trasformazione si chiama **currying**
esempio javascript

```
function somma(x, y) {  
  return x + y;  
}
```

con currying

```
function somma(x) {  
  return (y => x + y);  
}
```

let $f = \text{somma}(3) \rightarrow$ questo restituisce una funzione con x fissato a 3
ora f è una funzione $\rightarrow f(5)$ in questo modo fissiamo anche y
e questo punto ritorna un valore = 8

Passaggio dei parametri

nel λ -calcolo quando ho una funzione applicata a qualcosa

$(\lambda x.x) (\lambda y.y) z$ • Posso scegliere quale tra i 2 redex
Valutare prima
Redex

Invece nei linguaggi di programmazione quando si chiama una funzione passando un'espressione, si risolve prima l'espressione e il risultato si passa alla funzione
es: $f(3+2) \rightarrow f(5) \rightarrow \text{risultato}$

- Per rendere il λ -calcolo più simile ai linguaggi di programmazione

- Regole per valutazione dell'applicazione di funzione:

1° Passo: valutazione dell'espressione che definisce la funzione
quindi non scelgo come redex il corpo di una funzione

2° Passo:

1° call-by-value (CBV): prima di passare qualcosa alla funzione
valuto quello che devo passare

esempio:

$(\lambda x.x) ((\lambda y.y) z) \rightarrow (\lambda x.x) z \rightarrow z$
prima valuto poi valuto
cioè che devo la funzione
passare (argomento)

- Valuto prima il redex intermo

2° coll - by-name (CBN): se ho un'applicazione con dei redex da fare non vedo e valuto il valore dell'argomento, lo riscrivo tutto nel corpo della funzione (il contrario dell'altra)

- scelgo il redex più esterno

Formalizziamo le Beta riduzioni

(senza strategie) = qualsiasi redex può essere ridotto in qualsiasi momento

$$(\lambda x. e_1) e_2 \rightarrow e_1 \{x := e_2\}$$

$$\frac{1^\circ \quad e_1 \rightarrow e'_1}{e_1 e_2 \rightarrow e'_1 e_2}$$

$$\text{Beta red} \rightarrow \frac{\overset{e_1}{(\lambda x. x)} (\overset{e'_1}{\lambda z. z}) \rightarrow (\lambda z. z)}{\underbrace{((\lambda x. x) (\lambda z. z))}_K \rightarrow \underbrace{(\lambda z. z)}_K}$$

$\underbrace{\hspace{10em}}_{e_1} \quad \underbrace{\hspace{10em}}_{e_2}$

$$\frac{2^\circ \quad e_2 \rightarrow e'_1}{e_1 e_2 \rightarrow e_1 e'_1}$$

$$\frac{\overset{e_2}{(\lambda z. z)}_K \rightarrow \overset{e'_1}{K}}{\underbrace{((\lambda x. x) ((\lambda z. z)_K))}_{e_1} \rightarrow \underbrace{(\lambda x. x)_K}_{e'_1}}$$

$$\frac{3^\circ \quad e \rightarrow e'}{\lambda x. e \rightarrow \lambda x. e'}$$

$$\frac{(\lambda h. h)_z \rightarrow z}{\underbrace{(\lambda x. ((\lambda h. h)_z))}_{\lambda x. e} \rightarrow \lambda x. z}$$

Modifico le regole d'inferenza per le strategie

Call-by-Value

- Prima si calcola completamente l'argomento
Poi si applica la funzione

$$(\lambda x. e_1) v \rightarrow e_1 \{x := v\}$$

↓
espressione non
riducibile

$$\frac{e_1 \rightarrow e'}{e_1 e_2 \rightarrow e' e_2} \quad \frac{(\lambda x. x) k \rightarrow k}{\frac{((\lambda x. x) k)}{e_1} z \rightarrow k z} \quad e_2$$

$$\frac{e_2 \rightarrow e'}{v e_2 \rightarrow v e'}$$

Call-by-name

- non si valuta l'argomento prima della chiamata
si applica la funzione e ~~poi~~ possibile (la mettendola in forma $\lambda x. e_1$)

$$(\lambda x. e_1) e_2 \rightarrow e_1 \{x := e_2\}$$

$$\frac{e_1 \rightarrow e'}{e_1 e_2 \rightarrow e' e_2}$$

$$(\lambda x. x) ((\lambda y. y) k) \rightarrow (\lambda y. y) k$$

- vale la confluenza quindi non importa come lo esegui, il risultato sarà lo stesso

- usando le strategie però non scendo nel corpo della funzione

esempio

$$* \rightarrow (\lambda x. ((\lambda k. k) z)) \rightarrow_{\text{senza strategia}} \lambda x. z$$

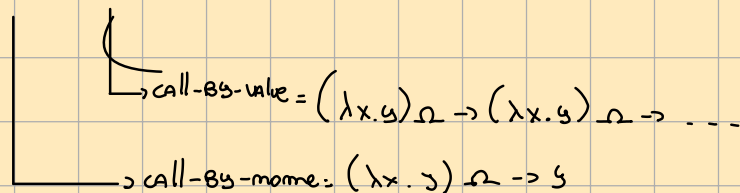
"

con strategie questo sarebbe il risultato

esempio

Ω = lambda espressione che ritorna in se stessa

- $(\lambda x. y) \Omega \rightarrow$ senza strategie avo terminare oppure no



e la ricorsione?

- non sono previsti meccanismi primitivi nel λ -calcolo
- il problema è che non ho nomi, ho funzioni anonime

Il combinateur Y non ha variabili libere

$Y = \lambda f. (\lambda x. f(x x)) (\lambda x. f(x x)) \rightarrow$ assomiglia a Ω cioè un ciclo infinito però mette al suo interno f

Y permette di chiamare se stesso in modo implicito

$$Y F \equiv_{\beta} F(\underbrace{(Y F)}_{F(Y F)}) \equiv F(F(Y F))$$

esempio

$Y F = \lambda f. (\lambda x. f(x x)) (\lambda x. f(x x))$ $F \rightarrow$ funzione da copiare

$$\rightarrow (\lambda x. F(x x)) (\lambda x. F(x x))$$

$$\rightarrow F((\lambda x. F(x x)) (\lambda x. F(x x)))$$

$$\rightarrow F(F((\lambda x. F(x x)) (\lambda x. F(x x))))$$

$\rightarrow \dots$

esempio fattoriale

Funzione
Ricorsiva $G = \lambda f. \lambda m. \underbrace{if(m=0) then 1 else m * f(m-1)}_{\text{serve per fare la funzione ricorsiva}} \rightarrow$ Funzione da copiare

$Y G =$ so G Ad Y e mi fa la Ricorsione